

System Analizy Ruchu Drogowego

Automatyczne zliczanie i klasyfikacja pojazdów

Analiza Obrazów

Eryk Kurdziel, Radosław Mieńkowski,
Kamil Krawiec, Mateusz Sieniawski, Damian Mucha

25 Stycznia 2026

Repozytorium projektu:

<https://github.com/KamilKr1355/TrafficAnalysis>

Spis treści

1	Opis projektu	3
1.1	Założenia aplikacji	3
2	Architektura aplikacji	3
2.1	Przegląd architektury	3
2.2	Algorytmy i techniki	3
2.2.1	Detekcja ruchu – Background Subtraction (MOG2)	3
2.2.2	Klasyfikacja pojazdów – YOLOv8	4
2.2.3	Śledzenie obiektów – Centroid Tracking	4
2.2.4	Zliczanie kierunkowe – Progi przejścia	4
2.3	Użyte biblioteki	5
3	Instrukcja uruchomienia	5
3.1	Wymagania systemowe	5
3.2	Instalacja i uruchomienie programu	5
4	Podział pracy w zespole	6
5	Dane wejściowe	6
5.1	Źródła danych	6
5.2	Format danych wejściowych	6
5.3	Dane dostarczone z projektem	6
6	Wyniki i format wyjściowy	6
6.1	Pliki wyjściowe	6
6.1.1	summary.csv – Podsumowanie statystyk	7
6.1.2	events.csv – Log zdarzeń	7
6.1.3	trajectories.png – Wizualizacja trajektorii	7
7	Ograniczenia i znane problemy	7
7.1	Kiedy program nie działa poprawnie	7
7.2	Wymagania dla poprawnego działania	7
7.3	Możliwe błędy i rozwiązania	8
7.3.1	Problem: Brak wykrywania pojazdów	8
7.3.2	Problem: Plik wideo nie został znaleziony	8
7.4	Problemy z działaniem na różnych nagraniach	8
8	Podsumowanie	9

1 Opis projektu

1.1 Założenia aplikacji

System Analizy Ruchu Drogowego służy do automatycznego wykrywania, śledzenia i klasyfikacji pojazdów na nagraniach wideo. Program analizuje materiał wideo z kamer monitoringu i generuje statystyki dotyczące natężenia ruchu w czterech kierunkach.

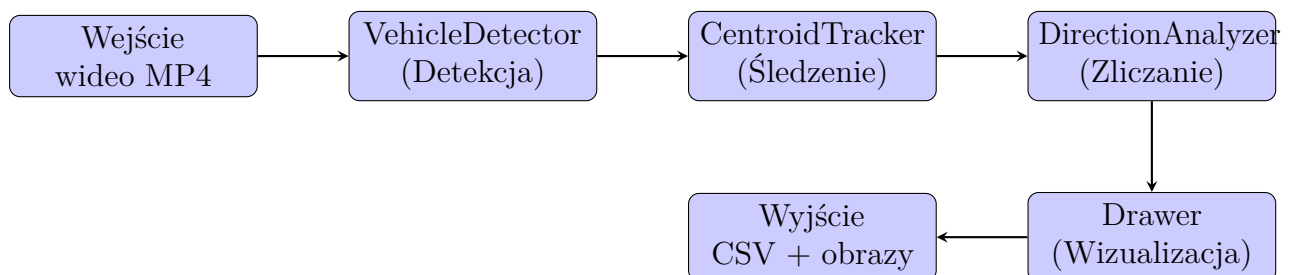
Główne funkcjonalności:

- Wykrywanie poruszających się pojazdów w czasie rzeczywistym
- Śledzenie trajektorii poszczególnych pojazdów
- Klasyfikacja pojazdów na 4 kategorie: osobowe, ciężarowe, autobusy, jednoślady
- Zliczanie pojazdów przejeżdżających w kierunkach: góra, dół, lewo, prawo
- Generowanie raportów CSV i wizualizacji trajektorii przejazdu

2 Architektura aplikacji

2.1 Przegląd architektury

System składa się z czterech głównych modułów przetwarzających dane:



Rysunek 1: Diagram przepływu danych

2.2 Algorytmy i techniki

2.2.1 Detekcja ruchu – Background Subtraction (MOG2)

Do wykrywania poruszających się obiektów wykorzystujemy algorytm **MOG2** (Mixture of Gaussians) z biblioteki OpenCV.

Parametry użyte w projekcie:

```
1 cv2.createBackgroundSubtractorMOG2(  
2     history=500,          # Liczba klatek historii  
3     varThreshold=50,      # Prog czułości  
4     detectShadows=True   # Wykrywanie cieni  
5 )
```

Listing 1: Konfiguracja MOG2

2.2.2 Klasyfikacja pojazdów – YOLOv8

Do rozpoznawania typu pojazdu używamy sieci neuronowej YOLOv8 z biblioteki Ultralytics.

Charakterystyka YOLO:

- Detekcja obiektów w jednym przebiegu sieci (real-time)
- Pretrenowany model na zbiorze COCO (80 klas pojazdów)
- Wyjście: bounding box + klasa pojazdu + pewność identyfikacji

Mapowanie klas COCO na kategorie systemu:

ID COCO	Klasa oryginalna	Nasza kategoria
1	bicycle	JEDNOSŁAD
2	car	OSOBOWY
3	motorcycle	JEDNOSŁAD
5	bus	BUS
7	truck	CIEZAROWY

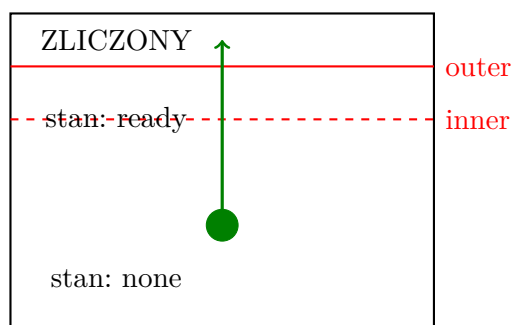
Tabela 1: Mapowanie klas

2.2.3 Śledzenie obiektów – Centroid Tracking

Algorytm śledzenia oparty na odległości centroidów.

2.2.4 Zliczanie kierunkowe – Progi przejścia

System wykorzystuje pary linii dla każdego kierunku:



Rysunek 2: Mechanizm zliczania (kierunek UP)

2.3 Użyte biblioteki

Biblioteka	Wersja	Zastosowanie
OpenCV	4.8+	Przetwarzanie obrazu, MOG2, operacje morfologiczne
Ultralytics	8.0+	Model YOLOv8 do klasyfikacji pojazdów
NumPy	1.24+	Operacje na tablicach, obliczenia matematyczne
Pandas	2.0+	Przechowywanie trajektorii, eksport CSV
Matplotlib	3.7+	Generowanie wykresów i wizualizacji

Tabela 2: Zależności projektu

3 Instrukcja uruchomienia

3.1 Wymagania systemowe

- Python 3.8 lub nowszy
- System: Windows / Linux / macOS

3.2 Instalacja i uruchomienie programu

```
1 git clone https://github.com/KamilKr1355/TrafficAnalysis.git
2 cd traffic-analysis
3
4
5 pip install -r requirements.txt
6
7 # Plik wideo należy umieścić w data/videos/video.mp4
8 # Uruchomienie programu:
9 python -m src.main
```

Listing 2: Procedura uruchomienia programu

4 Podział pracy w zespole

Osoba	Zakres odpowiedzialności
Eryk Kurdziel	Moduł <code>VehicleDetector</code> – implementacja detekcji MOG2, integracja YOLOv8, klasyfikacja pojazdów
Radosław Mieńkowski	Moduł <code>CentroidTracker</code> – algorytm śledzenia, zarządzanie ID pojazdów, struktura <code>DataFrame</code>
Kamil Krawiec	Moduł <code>DirectionAnalyzer</code> – logika zliczania, maszyna stanów, detekcja przekroczeń linii
Mateusz Sieniawski	Moduł <code>Drawer</code> – wizualizacja, interfejs HUD, generowanie wykresów trajektorii
Damian Mucha	Integracja systemu, plik <code>main.py</code> , konfiguracja, testy, dokumentacja

Tabela 3: Podział zadań w zespole

5 Dane wejściowe

5.1 Źródła danych

Do testowania systemu wykorzystano następujące źródła nagrań wideo:

1. Nagrania z monitoringu miejskiego
2. Pexels <https://www.pexels.com/search/videos/traffic/>

5.2 Format danych wejściowych

- **Format:** MP4
- **Kamera:** Nieruchoma, umocowana stale w jednym miejscu

5.3 Dane dostarczone z projektem

W repozytorium znajduje się przykładowy plik testowy:

```
data/videos/video.mp4 -- przykładowe nagranie ruchu drogowego  
dla którego detekcja przebiega prawidłowo
```

6 Wyniki i format wyjściowy

6.1 Pliki wyjściowe

System generuje następujące pliki w katalogu `output/`:

6.1.1 summary.csv – Podsumowanie statystyk

Kierunek	OSOBOWY	CIEZAROWY	BUS	JEDNOSLAD
UP	45	12	3	8
DOWN	52	8	5	12
LEFT	23	4	1	3
RIGHT	31	6	2	5

6.1.2 events.csv – Log zdarzeń

ID	Typ	Kierunek	Czas [s]
0	OSOBOWY	UP	1.23
1	CIEZAROWY	DOWN	2.45
2	BUS	LEFT	3.67

6.1.3 trajectories.png – Wizualizacja trajektorii

Obraz z nałożonymi ścieżkami wszystkich wykrytych pojazdów.

7 Ograniczenia i znane problemy

7.1 Kiedy program nie działa poprawnie

Problem	Przyczyna	Objawy
Ruchoma kamera	MOG2 wymaga statycznego tła	Całe tło wykrywane jako ruch
Bardzo gęsty ruch	Nachodzące się pojazdy	Błędne przypisania ID, gubienie śladów
Pojazdy częściowo zasłonięte	Niepełny kontur	Błędna klasyfikacja lub brak detekcji
Bardzo małe obiekty	Filtracja min_area	Rowery/skutery mogą być pomijane
Niski FPS wideo	Za duże przesunięcia między klatkami	Utrata ciągłości śledzenia

Tabela 4: Znane ograniczenia systemu

7.2 Wymagania dla poprawnego działania

1. Kamera musi być statyczna (brak drgań, ruchu)
2. Nagranie powinno mieć stabilne oświetlenie

3. Pojazdy nie powinny się znacząco zasłaniać
4. FPS minimum 15 klatek/sekundę
5. Rozdzielczość minimum 640x480

7.3 Możliwe błędy i rozwiązania

Poniżej przedstawiono najczęściej występujące problemy podczas użytkowania systemu oraz sposoby ich rozwiązania.

7.3.1 Problem: Brak wykrywania pojazdów

Objawy: Program działa, wideo jest wyświetlane, ale pojazdy nie są zliczane pomimo ich widocznego ruchu w kadrze.

Przyczyna: Linie zliczające są ustawione w nieodpowiednich miejscach - pojazdy mogą przejeżdżać poza obszarem wyznaczonym przez linie lub linie mogą być ustawione w złej kolejności.

Rozwiązanie: Należy dostosować położenie linii zliczających poprzez modyfikację parametrów `out` oraz `inn` w pliku `main.py`. Linie te zdefiniowane są w strukturze `lines_config`, która określa granice zliczania dla czterech kierunków ruchu.

Parametr `inn` (linia wewnętrzna) wyznacza strefę gotowości pojazdu do zliczenia, natomiast parametr `out` (linia zewnętrzna) odpowiada za moment zliczenia pojazdu.

Wskazówka: Uruchom program w trybie podglądu i zaobserwuj, w których miejscach kadru poruszają się pojazdy. Linia `inner` powinna być ustawiona wcześniej na trasie pojazdu, a `outer` dalej w kierunku ruchu.

7.3.2 Problem: Plik wideo nie został znaleziony

Komunikat błędu:

Error: Video file not found

Przyczyna: Program nie może odnaleźć pliku wideo w oczekiwanej lokalizacji.

Rozwiązanie: Należy upewnić się, że plik wideo znajduje się w katalogu `data/videos/`

7.4 Problemy z działaniem na różnych nagraniach

Nie wszystkie nagrania działają poprawnie w systemie zliczania pojazdów. Główne powody to:

- **Nietypowe kąty widzenia:** W nagraniach z nietypowego kąta kamery pojazdy mogą częściowo wychodzić poza kadr lub zostać ucięte. Skutkiem może być brak zliczenia lub błędna kategoryzacja pojazdu.
- **Efekt perspektywy:** W nagraniach, gdzie dół kadru znajduje się bliżej kamery, a góra dalej, rozmiary pojazdów w pikselach się różnią. W konsekwencji dalsze pojazdy mogą nie przekraczać progu `MIN_CONTOUR_AREA`, przez co nie są zliczane.

- **Bliskość i duże obiekty:** W nagraniach, w których pojazdy znajdują się bardzo blisko kamery lub zajmują dużą część kadru, **MOG2** może wykrywać wiele fragmentów jednego pojazdu jako osobne obiekty. W efekcie przypisywane są im różne ID, co skutkuje wielokrotnym zliczaniem tego samego pojazdu lub przypisaniem mu błędnej kategorii.

8 Podsumowanie

Projekt realizuje zadanie wykrywania i analizy obiektów na obrazie wideo. System łączy klasyczne metody przetwarzania obrazu (odejmowanie tła, morfologia) z głębokim uczeniem (YOLOv8).